Python Strings

Creating Strings

In Python, strings can be created in different ways:

- **Using double quotes**

```
name = "john"
grade = "B"
```

- **Using single quotes**

```
name = 'john'
grade = 'B'
```

- **Using str() constructor**

```
name = str()      # empty string
grade = str()     # empty string

name = str("David")
grade = str("B")

print(name)       # David
print(type(name))  # <class 'str'>
```

Note: Strings are always objects of the class str.

String Operators

- + → Concatenates (joins) strings
- * → Repeats a string multiple times

```
s = "welcome"
print(s + " programming")   # welcome programming
print(s * 3)                # welcomewelcomewelcome
```

Slicing Strings

- Strings can be sliced using indexes.
- Index starts from **0** (left → right).
- Negative index counts from **end** (right → left).

Example 1

```
mystr = "welcome"
print(mystr[1:3])    # el
print(mystr[:6])     # welcom
print(mystr[2:])     # lcome
print(mystr[1:-1])   # elcom
print(mystr[-5:-2])  # lco
```

Example 2

```
s = "abcdefghijklmno"
print(s[-4:])      # lmno
print(s[:-3])      # abcdefghijkl
print(s[-5:-2])    # klm
print(s[-8:-1:2])  # hjln
```

Explanation:

- s[-4:] → from 4th char from end (m) till end → "lmno"
- s[:-3] → from start till 3rd char from end (k excluded) → "abcdefghijkl"
- s[-5:-2] → from 5th char from end (l) till 2nd from end (excluded) → "klm"
- s[-8:-1:2] → from 8th char from end (g) to 2nd from end, step 2 → "hjln"

Reverse a String

Use a negative step (-1).

```
s = "Python"
print(s[::-1])  # nohtyP
```

Note: This creates a reversed copy. The original string remains unchanged.

Formatting Strings

- The **f-string** (Python 3.6+) is recommended.

```
age = 30
txt = f"My name is John, I am {age}"
print(txt)   # My name is John, I am 30

price = 59
print(f"The price is {price} dollars")      # The price is 59 dollars
print(f"The price is {price:.2f} dollars")   # The price is 59.00 dollars
print(f"The price is {20 * 59} dollars")     # The price is 1180 dollars
```

Old method: "Hello, {}".format(name) still works but f-strings are shorter.

Membership Operators

- in → Check if substring exists
- not in → Check if substring does not exist

```
s = "welcome"
print("come" in s)      # True
print("lome" in s)      # False
print("come" not in s)   # False
print("lome" not in s)   # True
```

Strings are Immutable

- **Mutable object** → Can be changed after creation (list, dict, set).
- **Immutable object** → Cannot be changed after creation (int, float, tuple, str).

Example: Immutability

```
s1 = "hello"
print("Original String:", s1)

s1[0] = "H"   # ❌ Error - strings are immutable
Instead, create a new string:
s2 = "H" + s1[1:]
print("Modified String:", s2)
Check object IDs:
print("s1 id:", id(s1))
print("s2 id:", id(s2))   # different id → new object created
```

Example: Mutable Object (list)

```
lst = ["h", "e", "l", "l", "o"]
print("Original List:", lst)

lst[0] = "H"   # ✅ works
print("Modified List:", lst)
print("lst id:", id(lst))  # same id → modified in place
```

Summary:

- String → immutable (cannot be modified in place).
- List, dict, set → mutable.

String Length

len()- returns the length of a string

```
a = "Welcome"
print(len(a))  # 7
```

String Methods

(a) Case Conversion

```
s = "hello"
print(s.capitalize())  # Hello

s = "HeLLo"
print(s.casefold())    # hello

print("Hello".lower())  # hello
print("Hello".upper())  # HELLO
print("welcome to python".title())  # Welcome To Python
print("Welcome To Python".swapcase())  # wELCOME tO pYTHON
```

(b) Alignment & Formatting

```
print("Hi".center(6, '*'))      # **Hi**
print("Hello, {}!".format("John"))  # Hello, John!
```

(c) Search & Find

```
s = "hello"
print(s.find("l"))   # 2
print(s.find("x"))   # -1
print(s.index("e"))  # 1
# print(s.index("x"))  # ❌ Error
```

(d) Count & Replace

```
s = "banana"
print(s.count("a"))   # 3
s = "Hello World"
print(s.replace("World", "There"))  # Hello There
```

(e) Validation (Boolean Checks)

```
print("abc123".isalnum())   # True
print("abc!".isalnum())     # False
print("Hello".isalpha())    # True
print("hello".islower())    # True
print("HELLO".isupper())    # True
```

(f) isdecimal(), isdigit(), isnumeric()

- **isdecimal()**
  - Returns **True** if all characters in the string are **decimal characters (0–9 only)**.
  - These are strictly digits used to form base-10 numbers.
  - Does **not** consider superscripts, fractions, or Roman numerals as valid decimals.

  Example:

  ```
  print("123".isdecimal())   # True (all decimal digits)
  print("①②③".isdecimal())   # False (circled numbers are not decimal)
  print("10.5".isdecimal())  # False (dot is not decimal)
  ```

- **isdigit()**
  - Returns **True** if all characters are **digits**.
  - Digits include **decimal digits (0–9) plus** other digit characters, such as superscripts, subscript digits, or digits from other numeral systems.

  Example:

  ```
  print("123".isdigit())     # True (decimal digits)
  print("²³".isdigit())      # True (superscript 2 and 3)
  print("⑤".isdigit())       # True (Japanese/Chinese digit in a circle)
  print("10.5".isdigit())    # False (dot not allowed)
  ```

- **isnumeric()**
  - Returns **True** if all characters are numeric.

- This is the **broadest check** — includes **digits, decimals, vulgar fractions, Roman numerals, currency numerators, etc.**
- Accepts everything isdecimal() and isdigit() accept, plus additional numeric characters.

Example:

```
print("123".isnumeric())   # True (decimal digits)
print("²³".isnumeric())    # True (superscripts)
print("⑤".isnumeric())     # True (circled digit)
print("⅔".isnumeric())     # True (fraction two-thirds)
print("Ⅷ".isnumeric())     # True (Roman numeral 8)
print("10.5".isnumeric())  # False (dot not allowed)
```

Method	Accepts Only 0–9	Superscripts (², ⑤)	Fractions & Roman (⅔, Ⅷ)
isdecimal()	✅ Yes	❌ No	❌ No
isdigit()	✅ Yes	✅ Yes	❌ No
isnumeric()	✅ Yes	✅ Yes	✅ Yes

- Use isdecimal() for base-10 digits only.
- Use isdigit() for digits + superscripts.
- Use isnumeric() for the broadest check.

(g) Splitting & Joining

```
s = "one,two,three"
print(s.split(","))   # ['one', 'two', 'three']
```

(h) Start & End Checks

```
s = "hello"
print(s.startswith("he"))  # True
s = "hello.py"
print(s.endswith(".py"))   # True
```

(i) Trimming Whitespaces

```
print("  hello  ".strip())   # hello
print("  hello".lstrip())    # hello
print("hello  ".rstrip())    # hello
```

String Methods Summary

- **Case conversion** → capitalize(), casefold(), lower(), upper(), title(), swapcase()
- **Alignment & formatting** → center(), format()
- **Search & find** → find(), index()
- **Counting & replacing** → count(), replace()
- **Validation** → isalnum(), isalpha(), isdecimal(), isdigit(), isnumeric(), islower(), isupper()
- **Splitting** → split()
- **Start/End checks** → startswith(), endswith()
- **Trimming spaces** → strip(), lstrip(), rstrip()